SAARLAND UNIVERSITY
Prof. Jörg Hoffmann, Dr. Daniel Höller
Thorsten Klößner, Jan Eisenhut



# Project 2: Pattern Database Heuristics

In this project, your task is to implement the pattern database heuristic and the cliques heuristic. For a planning task $\Pi = (V, A, I, G)$ and a pattern (subset of variables) $P \subseteq V$, the pattern database (PDB) heuristic is the abstraction heuristic $h^P = h^{\pi_P}$, induced by the abstraction function $\pi_P(s) := s|_P$ on $\Theta_\Pi$, where $s|_P$ is the restriction of $s$ to $P$. The cliques heuristic for a set of patterns $\mathcal{C}$ is defined as

$$h^{\mathcal{C}}(s) := \max_{D \in \text{cliques}(\mathcal{C})} \sum_{P \in D} h^P(s),$$

where cliques($\mathcal{C}$) is the set of all maximal cliques in the compatibility graph of $\mathcal{C}$.

## 1 Obtaining the Project Files

Your remote repository was updated with the files of this project when the project started. Simply `git pull` the project to obtain them locally.

## 2 Documentation

The project framework is supplemented by an online documentation. Like the last two projects, this project deals with the implementation of heuristics, so recall the documentation of the planning task representation, the Heuristic class and the utility functions. However, there are also some new functionalities that you should take a look at before starting with the implementation.

**Array Indexing** To map between states of a task and array indices, we provide you with the helper class `StateEnumerator`. This class is important for the implementation of $h^*$.

**Representing $\Pi|_P$** `SyntacticProjection` implements a `ClassicalPlanningTask` that is the syntactic projection $\Pi|_P$ of another planning task $\Pi$ with respect to a pattern $P$. This helper class can also compute the abstract state $s|_P$ for a state $s$ of $\Pi$. It is important for the implementation of $h^P$ and $h^{\mathcal{C}}$.

**Maximum Cliques** The helper function `max_cliques::compute_max_cliques` calculates all maximal cliques in a graph. This is relevant for the cliques heuristic $h^{\mathcal{C}}$, as you need to compute cliques($\mathcal{C}$), the maximal cliques in the compatibility graph of $\mathcal{C}$.

# 3 Implementation (15 + 3 + 7 Points = 25 Points)

## 3.1 Implementation of $h^*$ (15 Points)

Before implementing $h^P$ or $h^{\mathcal{C}}$, recall that we have the following relationships:

$$h^P(s) = h^*_{\Pi|_P}(s|_P) \qquad h^{\mathcal{C}}(s) = \max_{D \in \text{cliques}(\mathcal{C})} \sum_{P \in D} h^*_{\Pi|_P}(s|_P)$$

For both heuristics, it is required to compute $h^*$ for one or more planning tasks, so you must implement this computation first. To implement the computation of $h^*$ for all states of a planning task, implement the function

```
std::vector<int> compute_hstar_table(
  const ClassicalPlanningTaskProxy& task_proxy,
  const StateEnumerator& enumerator);
```

in the file `src/downward/pdbs/hstar.cc`. Invoked with a planning task $\Pi$ and a state enumerator $e$ serving as a mapping from states to array indices, this function must return a table `h_star` of $h^*$ values such that $h^*(s)$ is equal to `h_star[e.get_index(s)]` for every state $s$ of $\Pi$. Represent infinite values with the constant `Heuristic::DEAD_END`.

### Implementation Hints

As a first step, reduce the problem of computing $h^*$ for a task $\Pi$ to a regression search. The idea is to start the regression from the goal states and compute their minimum distance to all other states. To this end, first explicitly construct a weighted directed graph $\mathcal{G} = (V, E)$ with vertices $V$ and integer-weighted edges $E \subseteq V \times \mathbb{N} \times V$ similar to the transition system of $\Pi$, but with edges that go into the opposite direction. We call this graph the *regression graph*. The vertices $V$ of this graph are the state indices $\{0, \ldots, |S| - 1\}$ of the task. Let $index(s)$ be the index of $s$ as given by `enumerator.get_index(s)`. The weighted edge relation is defined by

$$E = \{(index(s[\![a]\!]), c(a), index(s)) \mid s \in S, a \in A(s)\}.$$

Construct and store the edge relation as an adjacency list $L$, such that list entry $L(i)$ gives you the list of weighted successors $(wgt_1, j_1)$, $(wgt_2, j_2)$, ... for the state with index $i$, where $j_1$, $j_2$, ... are indices of the successor states.

After constructing the regression graph $\mathcal{G}$, use Dijkstra's algorithm on $\mathcal{G}$ to compute the minimum distance from the goal vertices, i.e., the indices of the goal states, to every

other vertex, which effectively computes a lookup table for $h^*$ in the specified format. While Dijkstra's algorithm assumes a single source vertex, you can extend it to multiple source vertices by initializing the search queue with all source vertices and initializing their distances with 0.

### Public Tests

The public tests for this exercise are located in the file

`src/tests/heuristic_tests/hstar_table_tests.cc`

The test suite for this exercise is named `HStarTableTestsPublic`. The test preset `hstar` was added for command line users.

## 3.2 Implementation of $h^P$ (3 Points)

To implement $h^P$, implement the function

```
std::shared_ptr<Heuristic> create_pdb_heuristic(
  std::shared_ptr<ClassicalPlanningTask> task,
  pdbs::Pattern pattern);
```

in the file `src/downward/heuristics/pdb_heuristic.cc`. Invoked with a planning task $\Pi$ and a pattern $P$, this function must return an instance `h` of `Heuristic` such that `h.compute_heuristic(s)` always returns the value $h^P(s)$ with respect to $\Pi$. The invocation of `h.compute_heuristic(s)` must have $\mathcal{O}(|P|)$ worst-case runtime complexity.

### Implementation Hints

On construction, create and store the syntactic projection $\Pi|_P$ using the helper class `SyntacticProjection`, as well as a `StateEnumerator` for $\Pi|_P$. Also on construction, use your implementation of $h^*$ to pre-compute and store the lookup table with the values $h^*_{\Pi_P}(s')$ for all abstract states $s'$ of $\Pi|_P$, so that a simple lookup in this table for the abstract state $s|_P$ of the input state $s$ suffices later to compute the heuristic value for $s$ (which yields the required runtime complexity). The class `SyntacticProjection` also has a method to compute the abstract state $s|_P$ for an input state $s$. `StateEnumerator` can translate $s|_P$ (not $s$!) to an index for the table lookup.

Recomputing the lookup table each time the heuristic is evaluated is not an option due to the required runtime complexity. Doing so will inevitably lead to timeouts in the tests.

### Public Tests

The public tests for this exercise are located in the file

`src/tests/heuristic_tests/pdb_tests.cc`

The test suite for this exercise is named `PDBTestsPublic`. The test preset `pdb_heuristic` was added for command line users.

### 3.3 Implementation of $h^{\mathcal{C}}$ (7 Points)

To implement $h^{\mathcal{C}}$, implement the function

```
std::shared_ptr<Heuristic> create_cliques_heuristic(
  std::shared_ptr<ClassicalPlanningTask> task,
  std::vector<pdbs::Pattern> patterns);
```

in the file `src/downward/heuristics/cliques_heuristic.cc`. Invoked with a planning task $\Pi$ and a set of patterns $\mathcal{C}$, this function must return an instance `h` of `Heuristic` such that `h.compute_heuristic(s)` always returns the value $h^{\mathcal{C}}(s)$ with respect to $\Pi$. The invocation of `h.compute_heuristic(s)` must have worst-case runtime complexity

$$\mathcal{O}(\min\{\sum_{P\in\mathcal{C}}|P|, \sum_{D\in\text{cliques}(\mathcal{C})}|D|\}).$$

#### Implementation Hints

When constructing this heuristic, first compute the lookup tables $h^*_{\Pi|_P}$ for each pattern $P \in \mathcal{C}$ in the same way as for $h^P$. Next, compute cliques($\mathcal{C}$) using the helper function `max_cliques::compute_max_cliques` so that you can iterate over the set of cliques when evaluating the heuristic.

When evaluating this heuristic, first compute an array containing $h^P(s)$ for each pattern $P \in \mathcal{C}$ in time $\mathcal{O}(\sum_{P\in\mathcal{C}}|P|)$. Afterwards, iterate over all cliques $D \in$ cliques($\mathcal{C}$) to compute $h^{\mathcal{C}}$. Since $h^P(s)$ can be retrieved from the array in time $\mathcal{O}(1)$, this should take time $\mathcal{O}(\sum_{D\in\text{cliques}(\mathcal{C})}|D|)$. This implementation will give you the desired runtime.

#### Public Tests

The public tests for this exercise are located in the file

`src/tests/heuristic_tests/cliques_heuristic_tests.cc`

The test suite for this exercise is named `CliquesHeuristicTestsPublic`. The test preset `cliques_heuristic` was added for command line users.

### 3.4 Further Remarks

Both factory functions for $h^P$ and $h^{\mathcal{C}}$ are already partially implemented, each returning an instance of a dedicated class that derives from `Heuristic`, which is defined inside the

respective source file. The easiest way to complete the implementations is to complete the definition of the constructor and the inherited methods of these classes (see `TODO` comments).

**Attention:** Your implementation may not alter any files other than the mentioned source files. Changes to any other file will be discarded on our test server and will thus not have any effect.

# 4 Grading

The implementations are evaluated using public tests, which you may execute and inspect at any time, and daily tests which run at least once per day when you push a change to your repository.

## 4.1 Public Tests

In order to obtain points for a heuristic, you need to pass all public tests for it. The public tests evaluate your heuristic on example states for a sample planning task and check whether the result matches the correct heuristic value for that state. For example, both heuristics should be goal-aware, i.e. return zero when evaluated on a goal state. The name of the tests should give you an idea what the test is testing for. The five sample problems used in the public tests are documented on our documentation page and reside in the `domains` subfolder of the `tests` module.

## 4.2 Daily Tests

Daily tests are run once a day, if you have changed one of the implementation files. The result of the daily tests is both mailed to your student mail address as well as displayed on your personal status page in the CMS. If all public tests are passed, you will receive points for your implementation according to the ratio of daily tests you passed, i.e. if you pass 20% of the daily tests for $h^*$, you receive $20\% * 15 = 3$ points.

**Remarks** Note that your project must build on the daily test server in order to run tests for it. If your project does not finish the build process successfully within a generous amount of time and memory, no daily tests will be run and you will be notified of the broken build. Additionally, each individual test is run with a generous time and memory limit.

## 4.3 Deadline

The deadline for this project is **December 18th, 14:00**. We will use the last revision that was pushed to the main branch of your repository as a basis for grading. We will not consider any other revisions.